

MODULE 15 · DOMAIN 3 - DATA OPERATIONS AND SUPPORT

Data quality, skew and sampling

AWS Glue Data Quality, DQDL rules, consistency checks, sampling techniques and the mechanics of skew.

DOMAIN 3 of 4 EXAM WEIGHT 22% MODULE 15 of 27

A pipeline that succeeds tells you the **machinery** worked. It says nothing about the **numbers**.

Quality on data already in the warehouse

- ◆ **Constraints in Redshift are not enforced**, as module ten established.
- ◆ **So uniqueness is your pipeline's job**, through MERGE or an explicit check.
- ◆ **Glue Data Quality can evaluate a catalog table**, including Redshift through the catalog.
- ◆ **Scheduled SQL assertions** are the low-technology alternative that always works.
- ◆ A primary key declaration in Redshift is a hint to the planner, not a guarantee.

ON THE EXAM

Module ten's unenforced constraints and this module's quality rules are the same gap seen from two directions.

The dimensions of data quality

DIMENSION	THE QUESTION IT ASKS
Completeness	Are the values that should be there actually present?
Uniqueness	Is anything duplicated that should not be?
Accuracy	Do the values reflect reality?
Validity	Do values conform to the expected format and range?
Consistency	Do related datasets agree with each other?
Freshness	Is the data as recent as it should be?

ON THE EXAM

Freshness is the one that catches teams out. A pipeline can produce perfect data that is three days old.

Rules are written in **DQDL**, and they run either on a catalog table or inside a job.

DQDL, in the parts that matter

- ◆ A **ruleset** groups rules, and attaches to a Data Catalog table. Up to two thousand rules.
- ◆ **IsComplete, IsUnique, ColumnValues, ColumnLength** and around thirty other rule types.
- ◆ **Composite rules** combine conditions with and and or.
- ◆ A **WHERE clause** applies a rule to a subset of rows.
- ◆ **CustomSql** for anything the built-in rule types cannot express.

ON THE EXAM

You will not write DQDL in the exam. Recognising that a declarative ruleset exists, rather than custom code, is the point.

Rule types worth recognising

RULE	WHAT IT ASSERTS
IsComplete	The column has a value in every row
IsUnique / Uniqueness	No duplicates, or uniqueness above a threshold
ColumnValues	Values fall within an allowed set or range
RowCount / RowCountMatch	The dataset size is as expected, or matches another
ReferentialIntegrity	Values exist in a reference dataset
Freshness	The most recent record is no older than a stated interval
CustomSql	Anything the built-in types cannot express

ON THE EXAM

You are not asked to write these. Recognising that ReferentialIntegrity and Freshness exist as rule types is the point.

Rules you do not have to write

- ◆ **Recommendations.** Glue analyses a dataset and proposes a starting ruleset.
- ◆ **Dynamic rules** compare against recent history rather than a fixed threshold.
- ◆ **Anomaly detection** uses machine learning over statistics gathered across runs.
- ◆ **Analyzers** gather statistics without asserting anything, feeding the detection.
- ◆ Static rules only catch what somebody thought to check.

ON THE EXAM

Anomaly detection catches the failures nobody predicted. A rule catches the failures somebody did.

Choosing a threshold

- ◆ **Demanding one hundred percent** on real-world data produces constant failures.
- ◆ **Set the threshold from observed history**, not from an aspiration.
- ◆ **Different thresholds per column**, because optionality differs.
- ◆ **A rule that always fails** gets ignored, exactly like a noisy alarm.
- ◆ **Tighten over time** as upstream quality genuinely improves.

ON THE EXAM

A quality rule nobody acts on is worse than no rule, because it creates the appearance of control.

A retailer's nightly pipeline loads order data into Amazon Redshift. Last month a source system began populating a currency column with an empty string instead of the currency code for a small share of records, and revenue reports were understated for eleven days before anyone noticed.

The team wants to catch this class of problem before bad data reaches the warehouse, and does not want to write and maintain custom validation code.

What should the data engineer implement?

- ☐ A CloudWatch alarm on the Glue job's failed run count
- ☐ AWS Glue Data Quality rules inside the ETL job, failing the job or quarantining rows when they do not pass
- ☐ A custom Python validation step in the Glue job that checks each column
- ☐ Increase the Redshift cluster size so the load completes before business hours

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Alarm on failed runs
The job did not fail. It succeeded eleven times and loaded bad data each time.
- C

 Custom Python validation
Would work, and is exactly the custom code they said they did not want to maintain.
- B

 Glue Data Quality rules in the job
IsComplete on the currency column catches empty values as data flows through, before the warehouse sees them.
- D

 A bigger cluster
Capacity for a correctness problem. Nothing here was slow.

THE SIGNAL

Eleven successful nights producing wrong numbers. Every failure-detection option is answering the wrong question.

Fail the job, or quarantine the rows?

FAIL ON VIOLATION

Stop everything

- Nothing downstream sees bad data
- A human decides what to do next
- Correct for financial and regulated data
- A single bad row blocks the whole load

QUARANTINE

Route the bad rows aside

- Good rows continue to their destination
- Bad rows kept for investigation
- Correct when partial data is useful
- Somebody must actually review the quarantine

THE TIEBREAKER

Module six's distinction again. A permanent problem needs routing; the choice is whether partial data has value.

Where quality checks belong

- ◆ **At ingestion**, so bad data never enters the lake at all.
- ◆ **Between layers**, so a curated table is only built from validated input.
- ◆ **Before publishing**, using the write, audit, publish pattern from module nine.
- ◆ **After publishing** means consumers saw the bad data first.
- ◆ **The earlier the check, the cheaper the fix** and the smaller the blast radius.

ON THE EXAM

Iceberg branches make the third option enforceable. Write to a branch, validate, merge only if it passes.

Consistency is about **agreement between datasets**, and it is the hardest dimension to check.

Checking that two datasets agree

- ◆ **Row counts** between source and target. The cheapest and bluntest check.
- ◆ **Aggregate comparison**, summing a numeric column on both sides.
- ◆ **ReferentialIntegrity in DQDL** checks that values exist in a reference dataset.
- ◆ **DatasetMatch** compares two datasets row by row on a key.
- ◆ **DMS data validation** does this for a migration, as module four covered.

ON THE EXAM

A count match is not a content match. Two datasets can have identical row counts and different values.

Finding records that are nearly the same

- ◆ **Exact duplicates** are easy. A uniqueness rule or a distinct catches them.
- ◆ **Near duplicates** are not. Two customer records with a typo in the address.
- ◆ **Glue FindMatches** is a machine learning transform trained on labelled examples.
- ◆ It learns what counts as the same entity for your data, rather than applying a fixed rule.
- ◆ No amount of SQL or exact matching solves fuzzy identity.

ON THE EXAM

Module two introduced FindMatches. This is the data quality problem it exists for.

A finance team reports that monthly revenue in the warehouse is consistently around one percent below the figure in the source order system. The Glue pipeline reports success every night and row counts between source and target match exactly.

The pipeline joins orders to a currency conversion reference table before loading. Some orders use currency codes that were added to the source system recently.

What is the most likely cause, and which check would have caught it?

- ☐ A Rows are being dropped by the join. A ReferentialIntegrity rule on the currency code would have caught it
- ☐ B The Glue job is under-provisioned and timing out partway through
- ☐ C Redshift is applying compression that loses precision on the revenue column
- ☐ D The row count check is sufficient and the source system figure must be wrong

◆ PAUSE AND DECIDE

What is the most likely cause, and which check would have caught it?

- A

 Join dropping rows, caught by ReferentialIntegrity
An inner join silently discards orders whose currency is missing from the reference table. A referential check names exactly that.
- B

 Job timing out
A timeout fails the job. This one succeeds every night and the counts match.
- C

 Compression losing precision
Redshift compression is lossless. It does not alter values.
- D

 The source must be wrong
A consistent one percent gap correlated with new currency codes is a pattern, not a source error.

THE SIGNAL

Counts match because the count was taken after the join. The rows disappeared before anyone counted them.

You cannot always check everything.
Sampling is how you check enough.

Sampling techniques

TECHNIQUE	HOW IT WORKS
Random	Every row has an equal chance. Simple and usually adequate
Stratified	Sample within each group, preserving proportions
Systematic	Take every nth row. Cheap, and biased if the data has a cycle
Cluster	Sample whole groups rather than individual rows
First N rows	Not sampling at all. It reflects arrival order, not the population

ON THE EXAM

The last row is the mistake, not a technique. Reading the first thousand rows samples whoever arrived first.

When sampling misleads

- ◆ **Rare events** may not appear in a sample at all, which is not evidence of absence.
- ◆ **Skewed distributions** are misrepresented by a small random sample.
- ◆ **Ordered data** sampled systematically can align with a cycle and mislead badly.
- ◆ **Uniqueness cannot be sampled.** A duplicate may sit outside the sample.
- ◆ **Completeness can be,** because a null rate estimates well from a sample.

ON THE EXAM

Some checks work on a sample and some do not. Uniqueness and referential integrity need the full dataset.

Sampling in practice on AWS

- ◆ **Athena TABLESAMPLE** reads a proportion of the data, reducing bytes scanned.
- ◆ **Query one partition** as a cheap proxy for exploring a partitioned dataset.
- ◆ **Crawlers sample files** to infer a schema, which is why classifiers matter.
- ◆ **DataBrew profiles a sample** by default, with a configurable size.
- ◆ **A sampled profile of skewed data** understates the skew it was meant to reveal.

ON THE EXAM

Athena bills bytes scanned, so sampling is a direct cost lever as well as a speed one.

Skew has appeared in **four services** in this course. It is one mechanism, wearing four names.

The same problem, four services

SERVICE	HOW SKEW APPEARS
Apache Spark on Glue or EMR	One task runs for an hour while the rest finish in minutes
Kinesis Data Streams	Some shards throttle while total throughput is well under capacity
Amazon Redshift	One slice holds most of a table. Adding nodes does not help
Amazon DynamoDB	A partition throttles while overall consumed capacity looks low

ON THE EXAM

In every row, adding capacity provably fails, because the concentration survives it.

Fixing skew

- ◆ **Choose a higher cardinality key** so values spread across more units.
- ◆ **Salt the hot key** by appending a random suffix, then aggregate in two stages.
- ◆ **Broadcast the small side** of a join, so no redistribution happens at all.
- ◆ **Filter the outlier** if it is not real data, such as a test account.
- ◆ **Adding capacity** is the one thing that never works.

ON THE EXAM

Salting trades a simple aggregation for a two-stage one. That is the cost of spreading a key artificially.

A Glue job aggregating web events by session identifier has slowed from twenty minutes to over two hours. Monitoring shows two tasks running for the full duration while the remaining hundred and eighteen finish within four minutes. Data volume has grown about ten percent since the job was written.

Investigation shows that a monitoring bot and a load-testing harness account for a large share of all events, both using a fixed session identifier.

What should the data engineer do?

- ☐ A Increase the number of workers from thirty to ninety
- ☐ B Exclude the bot and load-test session identifiers, and salt any remaining hot keys
- ☐ C Switch to memory-optimised R workers so the long tasks have more memory
- ☐ D Enable Glue Auto Scaling so capacity adjusts to the workload

◆ PAUSE AND DECIDE

What should the data engineer do?

A Triple the workers

*Sixty extra executors idle while the same two tasks grind on.
The hot keys still hash to two places.*

C Memory-optimised workers

*Helps a skewed task survive without spilling. It does not
stop one task being most of the work.*

B Exclude the synthetic sessions, then salt

*The bot and the harness are not real user data. Removing
them eliminates the skew at source; salting handles what
remains.*

D Auto Scaling

*Adds workers when there is pending parallelisable work.
Two long tasks offer nothing to parallelise.*

THE SIGNAL

Two tasks out of a hundred and twenty, and ten percent volume growth. The growth is a decoy; the distribution is the problem.

Preparing and cleaning data

- ◆ **AWS Glue DataBrew** profiles a dataset and reports what is actually in it.
- ◆ **Over two hundred and fifty transformations** for cleaning, without code.
- ◆ **Glue Studio data preparation** is the successor, and imports DataBrew recipes.
- ◆ **SageMaker Unified Studio** is named alongside them in the exam guide.
- ◆ **Recipes are repeatable**, so an exploration becomes a scheduled job.
- ◆ Aimed at analysts. An engineer would usually write the transformation.

ON THE EXAM

Profiling before transforming is the habit. You cannot write sensible rules about data you have not looked at.

Making quality visible

- ◆ **Results go to CloudWatch**, so an alarm can fire on a failing ruleset.
- ◆ **Scores per ruleset** track the percentage of rules passing over time.
- ◆ **EventBridge on a quality event** can trigger a remediation or a notification.
- ◆ **DQDL labels** tag rules by criticality, owner or compliance requirement.
- ◆ **Results in the catalog** so consumers can see whether a dataset is trustworthy.

ON THE EXAM

Labels arrived in January 2026. They let you filter to high-priority failures owned by a particular team.

A data platform team must profile a new two hundred billion row dataset before designing quality rules for it. They need to understand null rates per column, value distributions and approximate cardinality. They also need to confirm that a customer identifier column contains no duplicates.

A full scan of the dataset is expensive and the team wants to limit cost where they reasonably can.

What approach should the data engineer take?

- ☐ A Profile a random sample for distributions and null rates, and run the uniqueness check across the full dataset
- ☐ B Profile a random sample and check uniqueness on the same sample
- ☐ C Read the first million rows and profile those
- ☐ D Run a full profile of every column across the entire dataset

◆ PAUSE AND DECIDE

What approach should the data engineer take?

- A

 Sample for distributions, full scan for uniqueness
Null rates and distributions estimate well from a sample. Uniqueness cannot be sampled, because the duplicate may sit outside it.
- B

 Sample for everything
A sample showing no duplicates proves nothing about the rows you did not read.
- C

 The first million rows
Not a sample. It reflects arrival order, so it describes whoever came first rather than the population.
- D

 Full profile of everything
Correct and needlessly expensive. Most of these checks do not require it.

THE SIGNAL

Two requirements with different sampling properties. The answer splits them rather than treating them the same.

Quality as a contract

- ◆ **Producers own the quality of what they publish**, not the consumers who suffer it.
- ◆ **Rules encode the contract**, so the expectation is explicit rather than assumed.
- ◆ **Publish scores with the dataset**, so consumers can judge before depending on it.
- ◆ **Version rules alongside pipeline code**, as module seven argued for everything else.
- ◆ An expectation nobody wrote down is discovered by being violated.

ON THE EXAM

The subscription workflow from module twelve is where a consumer sees these scores before requesting access.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Job succeeds, totals are low	Rows dropped by a join, or values coerced to null
Counts match, values disagree	A count check where a content check was needed
Duplicates appear after a failure	A retry over a non-idempotent append
A few tasks run far longer than the rest	Skew on a key. Never a capacity problem
A profile looks wrong	Sampled from the start of the data rather than randomly
Rules pass, data is still wrong	The rules only check what somebody thought of

ON THE EXAM

The last row is why anomaly detection exists. Rules encode known failures, and the interesting ones are unknown.

Reading a data quality question quickly

- ◆ **Did anything fail?** If not, every failure-detection option is wrong.
- ◆ **Which dimension?** Completeness, uniqueness, validity, consistency or freshness.
- ◆ **Known or unknown failure?** Rules for known, anomaly detection for unknown.
- ◆ **Uneven load?** Skew, and capacity is never the answer.
- ◆ **Can this be sampled?** Distributions yes. Uniqueness and referential integrity no.

ON THE EXAM

Next: cost and performance optimisation closes Domain 3.